Notebook comida.yaml Gemini

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
!pip install ultralytics

# 2. Descompactar o dataset
# Removemos a pasta antiga antes para garantir que não haja arquivos corrompidos
!rm -rf /content/test
!unzip -q /content/drive/MyDrive/PI_V/test.zip -d /content/
```

```
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (3.10.0)
Requirement already satisfied: opencv-python>=4.6.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (4.13.0.
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (11.3.0)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (6.0.3)
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.32.4)
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.16.3)
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.10.0+cu128)
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (0.25.0+cu
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (5.9.5)
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.35.2)
Collecting ultralytics-thop>=2.0.18 (from ultralytics)
  Downloading ultralytics_thop-2.0.18-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultral
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytic
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultra
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultra
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultraly
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultral
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ul
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.12/dist-packages (from polars>=0.20.0->
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0-
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultra
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultra
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.25.
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ul
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (75.
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultral
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0-
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0-
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0-
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ult
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->u
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ul
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0-
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib>
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.8.0->ultr
Downloading ultralytics-8.4.34-py3-none-any.whl (1.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 38.4 MB/s eta 0:00:00
Downloading ultralytics_thop-2.0.18-py3-none-any.whl (28 kB)
Installing collected packages: ultralytics-thop, ultralytics
Successfully installed ultralytics-8.4.34 ultralytics-thop-2.0.18
```

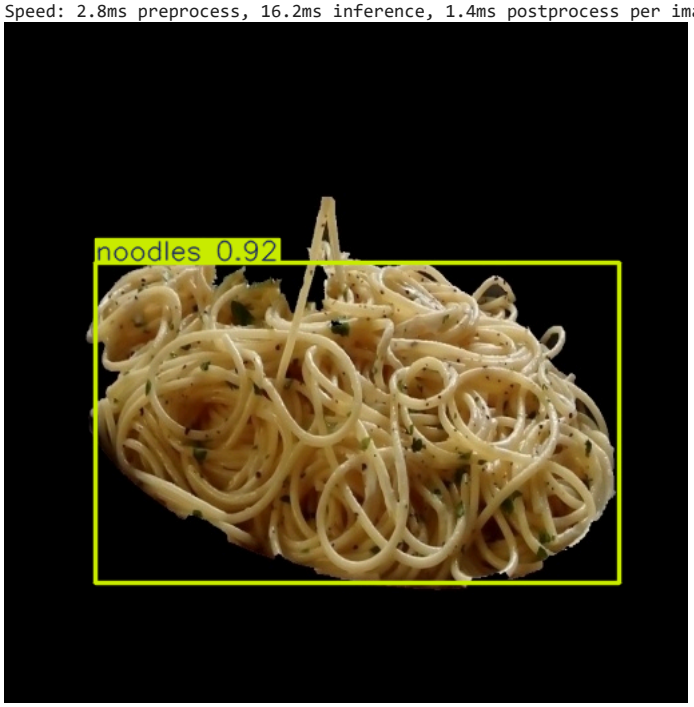
```
from ultralytics import YOLO

# Carrega o modelo 'Small' (yolov8s) - ele é muito melhor para identificar arroz e milho que o 'Nano'
model = YOLO('yolov8s.pt')

# Treina com 100 épocas para dar tempo do modelo aprender texturas claras (arroz/milho)
model.train(data='/content/comida.yaml', epochs=100, imgsz=640, device=0, name='treino_melhorado')
```

```
0.73774      0.73874      0.73974      0.74074      0.74174      0.74274      0.74374
```

```
0.75174,     0.75274,     0.75374,     0.74474,     0.74574,     0.74674,     0.74774,
            0.74474,     0.74575,     0.74675,     0.74775,     0.74875,     0.74975,     0.75075,     0.75175,
0.75275,     0.75375,     0.75475,     0.75576,     0.75676,     0.75776,     0.75876,     0.75976,     0.76076,
0.76176,     0.76276,     0.76376,     0.76476,     0.76577,     0.76677,     0.76777,
            0.76877,     0.76977,     0.77077,     0.77177,     0.77277,     0.77377,     0.77477,     0.77578,
0.77678,     0.77778,     0.77878,     0.77978,     0.78078,     0.78178,     0.78278,     0.78378,     0.78478,
0.78579,     0.78679,     0.78779,     0.78879,     0.78979,     0.79079,     0.79179,
            0.79279,     0.79379,     0.79479,      0.7958,      0.7968,      0.7978,      0.7988,      0.7998,
0.8008,      0.8018,      0.8028,      0.8038,      0.8048,     0.80581,     0.80681,     0.80781,     0.80881,
0.80981,     0.81081,     0.81181,     0.81281,     0.81381,     0.81481,     0.81582,
            0.81682,     0.81782,     0.81882,     0.81982,     0.82082,     0.82182,     0.82282,     0.82382,
0.82482,     0.82583,     0.82683,     0.82783,     0.82883,     0.82983,     0.83083,     0.83183,     0.83283,
0.83383,     0.83483,     0.83584,     0.83684,     0.83784,     0.83884,     0.83984,
            0.84084,     0.84184,     0.84284,     0.84384,     0.84484,     0.84585,     0.84685,     0.84785,
0.84885,     0.84985,     0.85085,     0.85185,     0.85285,     0.85385,     0.85485,     0.85586,     0.85686,
0.85786,     0.85886,     0.85986,     0.86086,     0.86186,     0.86286,     0.86386,
            0.86486,     0.86587,     0.86687,     0.86787,     0.86887,     0.86987,     0.87087,     0.87187,
0.87287,     0.87387,     0.87487,     0.87588,     0.87688,     0.87788,     0.87888,     0.87988,     0.88088,
0.88188,     0.88288,     0.88388,     0.88488,     0.88589,     0.88689,     0.88789,
            0.88889,     0.88989,     0.89089,     0.89189,     0.89289,     0.89389,     0.89489,      0.8959,
0.8969,      0.8979,      0.8989,      0.8999,      0.9009,      0.9019,      0.9029,      0.9039,      0.9049,
0.90591,     0.90691,     0.90791,     0.90891,     0.90991,     0.91091,     0.91191,
            0.91291,     0.91391,     0.91491,     0.91592,     0.91692,     0.91792,     0.91892,     0.91992,
0.92092,     0.92192,     0.92292,     0.92392,     0.92492,     0.92593,     0.92693,     0.92793,     0.92893,
0.92993,     0.93093,     0.93193,     0.93293,     0.93393,     0.93493,     0.93594,
            0.93694,     0.93794,     0.93894,     0.93994,     0.94094,     0.94194,     0.94294,     0.94394,
0.94494,     0.94595,     0.94695,     0.94795,     0.94895,     0.94995,     0.95095,     0.95195,     0.95295,
0.95395,     0.95495,     0.95596,     0.95696,     0.95796,     0.95896,     0.95996,
            0.96096,     0.96196,     0.96296,     0.96396,     0.96496,     0.96597,     0.96697,     0.96797,
0.96897,     0.96997,     0.97097,     0.97197,     0.97297,     0.97397,     0.97497,     0.97598,     0.97698,
0.97798,     0.97898,     0.97998,     0.98098,     0.98198,     0.98298,     0.98398,
            0.98498,     0.98599,     0.98699,     0.98799,     0.98899,     0.98999,     0.99099,     0.99199,
0.99299,     0.99399,     0.99499,       0.996,       0.997,       0.998,       0.999,           1]), array([[
1,           1,           1, ...,           0,           0,           0],
       [          1,           1,           1, ...,           0,           0,           0],
       [          1,           1,           1, ...,           0,           0,           0],
       ...,
       [          1,           1,           1, ...,           0,           0,           0],
       [          1,           1,           1, ...,           0,           0,           0],
       [          1,           1,           1, ...,           0,           0,           0]]), 'Confidence', 'Recall']]
fitness: 0.9751661607035039
keys: ['metrics/precision(B)', 'metrics/recall(B)', 'metrics/mAP50(B)', 'metrics/mAP50-95(B)']
maps: array([    0.98493,     0.99313,     0.96921,     0.95032,     0.97961,     0.94442,       0.995,     0.98028,
0.97778,     0.98534,     0.97869,     0.96329])
names: {0: 'beans', 1: 'beef', 2: 'broccoli', 3: 'carrot', 4: 'chicken', 5: 'corn', 6: 'egg', 7: 'noodles', 8: 'potato',
9: 'rice', 10: 'salad', 11: 'vegetables'}
nt_per_class: array([50, 49, 18, 44, 44,  7,  9, 10, 54, 68, 66, 37])
nt_per_image: array([50, 48, 18, 43, 44,  7,  9,  9, 51, 68, 64, 36])
results_dict: {'metrics/precision(B)': 0.994928513406594, 'metrics/recall(B)': 0.9781390736906338, 'metrics/mAP50(B)':
0.9949999999999998, 'metrics/mAP50-95(B)': 0.9751661607035039, 'fitness': 0.9751661607035039}
save_dir: PosixPath('/content/runs/detect/treino_melhorado4')
speed: {'preprocess': 0.240783754715833, 'inference': 5.478411018869784, 'loss': 0.0005523679239456025, 'postprocess':
5.966933518868915}
stats: {'tp': [], 'conf': [], 'pred_cls': [], 'target_cls': [], 'target_img': []}
```

```
from ultralytics import YOLO
from google.colab import files
from google.colab.patches import cv2_imshow

# Agora apontamos para a pasta 'treino_melhorado' que criamos acima
model = YOLO('/content/runs/detect/treino_melhorado4/weights/best.pt')

# Roda a validação detalhada
metrics = model.val()

print("\n" + "="*30)
print(f"RESUMO DAS MÉTRICAS:")
print(f"Precisão (Precision): {metrics.results_dict['metrics/precision(B)']:.4f}")
print(f"Revocação (Recall): {metrics.results_dict['metrics/recall(B)']:.4f}")
print(f"mAP50: {metrics.results_dict['metrics/mAP50(B)']:.4f}")
print("="*30)

# Bloco de Teste Prático (IMPORTANTE: Baixamos a confiança para 0.15)
print("Selecione a foto para testar (Arroz, Carne e Milho):")
uploaded = files.upload()
for fn in uploaded.keys():
    # conf=0.15 força o modelo a mostrar o que ele tem "dúvida", revelando o arroz e o milho
    results = model.predict(source=fn)
    res_plotted = results[0].plot()
    cv2_imshow(res_plotted)
```

```
Ultralytics 8.4.34 🚀 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 11,130,228 parameters, 0 gradients, 28.5 GFLOPs
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 2012.6±710.2 MB/s, size: 63.4 KB)
val: Scanning /content/train/labels.cache... 106 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 106/106 26.2Mit/s 0.0s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 7/7 1.5it/s 4.6s
                   all        106        456      0.995      0.978      0.995      0.975
                 beans         50         50      0.995          1      0.995       0.98
                  beef         48         49      0.997          1      0.995      0.993
              broccoli         18         18       0.99          1      0.995      0.969
                carrot         43         44      0.996          1      0.995       0.95
               chicken         44         44      0.999          1      0.995       0.98
                  corn          7          7          1      0.756      0.995      0.944
                   egg          9          9      0.979          1      0.995      0.995
               noodles          9         10      0.992          1      0.995       0.98
                potato         51         54      0.996          1      0.995      0.978
                  rice         68         68      0.997          1      0.995      0.985
                 salad         64         66      0.998          1      0.995      0.979
            vegetables         36         37          1      0.982      0.995      0.968
Speed: 7.2ms preprocess, 12.6ms inference, 0.0ms loss, 3.8ms postprocess per image
Results saved to /content/runs/detect/val

==============================
RESUMO DAS MÉTRICAS:
Precisão (Precision): 0.9949
Revocação (Recall): 0.9782
mAP50: 0.9950
==============================
Selecione a foto para testar (Arroz, Carne e Milho):
Escolher arquivos noodles0019.jpg
noodles0019.jpg(image/jpeg) - 50665 bytes, last modified: 01/04/2026 - 100% done
Saving noodles0019.jpg to noodles0019.jpg

image 1/1 /content/noodles0019.jpg: 640x640 1 noodles, 16.2ms
Speed: 2.8ms preprocess, 16.2ms inference, 1.4ms postprocess per image at shape (1, 3, 640, 640)
```



Comece a programar ou gere código com IA.

```
from ultralytics import YOLO
from google.colab import files
from google.colab.patches import cv2_imshow

# Agora apontamos para a pasta 'treino_melhorado' que criamos acima
model = YOLO('/content/runs/detect/treino_melhorado4/weights/best.pt')

# Roda a validação detalhada
metrics = model.val()

print("\n" + "="*30)
print(f"RESUMO DAS MÉTRICAS:")
print(f"Precisão (Precision): {metrics.results_dict['metrics/precision(B)']:.4f}")
print(f"Revocação (Recall): {metrics.results_dict['metrics/recall(B)']:.4f}")
print(f"mAP50: {metrics.results_dict['metrics/mAP50(B)']:.4f}")
print("="*30)

# Bloco de Teste Prático (IMPORTANTE: Baixamos a confiança para 0.15)
print("Selecione a foto para testar (Arroz, Carne e Milho):")
uploaded = files.upload()
```

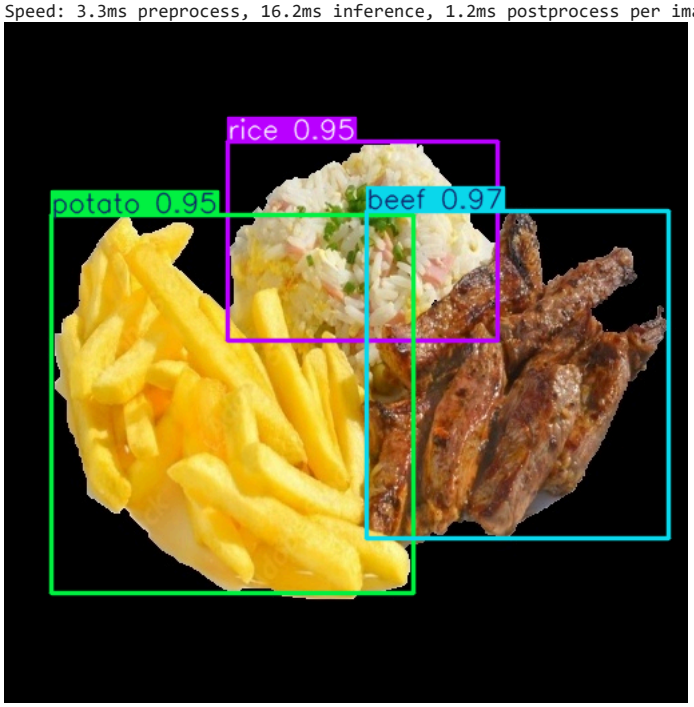
```
for fn in uploaded.keys():
    # conf=0.15 força o modelo a mostrar o que ele tem "dúvida", revelando o arroz e o milho
    results = model.predict(source=fn)
    res_plotted = results[0].plot()
    cv2_imshow(res_plotted)
```

```
Ultralytics 8.4.34 🚀 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 11,130,228 parameters, 0 gradients, 28.5 GFLOPs
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 2175.9±743.1 MB/s, size: 66.1 KB)
val: Scanning /content/train/labels.cache... 106 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 106/106 22.2Mit/s 0.0s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 7/7 1.3it/s 5.2s
                   all        106        456      0.995      0.978      0.995      0.975
                 beans         50         50      0.995          1      0.995       0.98
                  beef         48         49      0.997          1      0.995      0.993
              broccoli         18         18       0.99          1      0.995      0.969
                carrot         43         44      0.996          1      0.995       0.95
               chicken         44         44      0.999          1      0.995       0.98
                  corn          7          7          1      0.756      0.995      0.944
                   egg          9          9      0.979          1      0.995      0.995
               noodles          9         10      0.992          1      0.995       0.98
                potato         51         54      0.996          1      0.995      0.978
                  rice         68         68      0.997          1      0.995      0.985
                 salad         64         66      0.998          1      0.995      0.979
            vegetables         36         37          1      0.982      0.995      0.968
Speed: 7.6ms preprocess, 12.8ms inference, 0.0ms loss, 4.8ms postprocess per image
Results saved to /content/runs/detect/val2

==============================
RESUMO DAS MÉTRICAS:
Precisão (Precision): 0.9949
Revocação (Recall): 0.9782
mAP50: 0.9950
==============================
Selecione a foto para testar (Arroz, Carne e Milho):
```

Escolher arquivos fries0028.jpg

fries0028.jpg(image/jpeg) - 64043 bytes, last modified: 01/04/2026 - 100% done

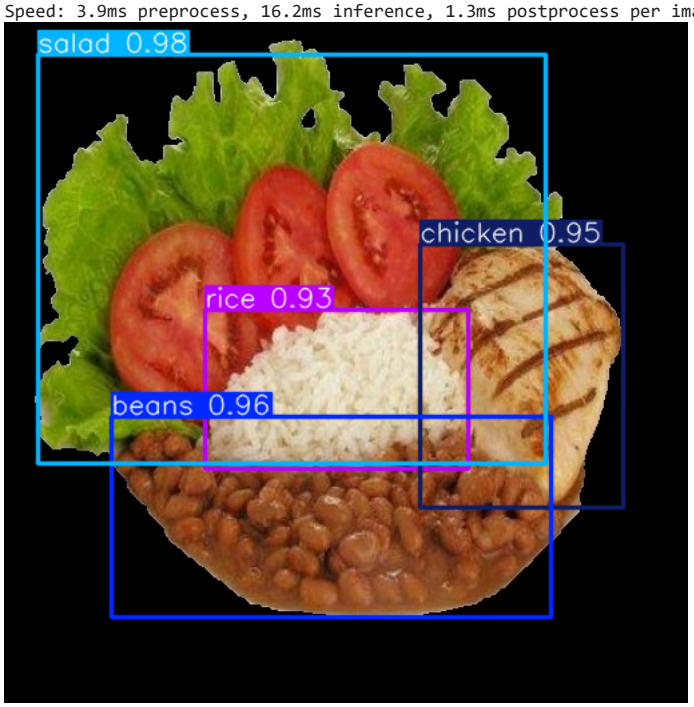
```
Saving fries0028.jpg to fries0028.jpg

image 1/1 /content/fries0028.jpg: 640x640 1 beef, 1 potato, 1 rice, 16.2ms
Speed: 3.3ms preprocess, 16.2ms inference, 1.2ms postprocess per image at shape (1, 3, 640, 640)
```




```
from ultralytics import YOLO
from google.colab import files
from google.colab.patches import cv2_imshow

# Agora apontamos para a pasta 'treino_melhorado' que criamos acima
model = YOLO('/content/runs/detect/treino_melhorado4/weights/best.pt')

# Roda a validação detalhada
metrics = model.val()

print("\n" + "="*30)
print(f"RESUMO DAS MÉTRICAS:")
print(f"Precisão (Precision): {metrics.results_dict['metrics/precision(B)']:.4f}")
print(f"Revocação (Recall): {metrics.results_dict['metrics/recall(B)']:.4f}")
print(f"mAP50: {metrics.results_dict['metrics/mAP50(B)']:.4f}")
print("="*30)
```

```
# Bloco de Teste Prático (IMPORTANTE: Baixamos a confiança para 0.15)
print("Selecione a foto para testar (Arroz, Carne e Milho):")
uploaded = files.upload()
for fn in uploaded.keys():
    # conf=0.15 força o modelo a mostrar o que ele tem "dúvida", revelando o arroz e o milho
    results = model.predict(source=fn)
    res_plotted = results[0].plot()
    cv2_imshow(res_plotted)
```

```
Ultralytics 8.4.34 🚀 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 11,130,228 parameters, 0 gradients, 28.5 GFLOPs
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 2481.9±742.6 MB/s, size: 83.4 KB)
val: Scanning /content/train/labels.cache... 106 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 106/106 44.5Mit/s 0.0s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 7/7 1.7it/s 4.2s
                   all        106        456      0.995      0.978      0.995      0.975
                 beans         50         50      0.995          1      0.995       0.98
                  beef         48         49      0.997          1      0.995      0.993
              broccoli         18         18       0.99          1      0.995      0.969
                carrot         43         44      0.996          1      0.995       0.95
               chicken         44         44      0.999          1      0.995       0.98
                  corn          7          7          1      0.756      0.995      0.944
                   egg          9          9      0.979          1      0.995      0.995
               noodles          9         10      0.992          1      0.995       0.98
                potato         51         54      0.996          1      0.995      0.978
                  rice         68         68      0.997          1      0.995      0.985
                 salad         64         66      0.998          1      0.995      0.979
            vegetables         36         37          1      0.982      0.995      0.968
Speed: 6.9ms preprocess, 11.9ms inference, 0.0ms loss, 3.3ms postprocess per image
Results saved to /content/runs/detect/val4

==============================
RESUMO DAS MÉTRICAS:
Precisão (Precision): 0.9949
Revocação (Recall): 0.9782
mAP50: 0.9950
==============================
Selecione a foto para testar (Arroz, Carne e Milho):
Escolher arquivos rice0030_jp...HYX8PL.jpg
rice0030_jpg.rf.GaeOsah6oTcGZrHYX8PL.jpg(image/jpeg) - 72450 bytes, last modified: 07/04/2026 - 100% done
Saving rice0030_jpg.rf.GaeOsah6oTcGZrHYX8PL.jpg to rice0030_jpg.rf.GaeOsah6oTcGZrHYX8PL.jpg

image 1/1 /content/rice0030_jpg.rf.GaeOsah6oTcGZrHYX8PL.jpg: 640x640 1 beans, 1 chicken, 1 rice, 1 salad, 16.2ms
Speed: 3.9ms preprocess, 16.2ms inference, 1.3ms postprocess per image at shape (1, 3, 640, 640)
```




```
from ultralytics import YOLO
from google.colab import files
from google.colab.patches import cv2_imshow

# Agora apontamos para a pasta 'treino_melhorado' que criamos acima
model = YOLO('/content/runs/detect/treino_melhorado4/weights/best.pt')

# Roda a validação detalhada
metrics = model.val()

print("\n" + "="*30)
print(f"RESUMO DAS MÉTRICAS:")
print(f"Precisão (Precision): {metrics.results_dict['metrics/precision(B)']:.4f}")
```

```
print(f"Revocação (Recall): {metrics.results_dict['metrics/recall(B)']:.4f}")
print(f"mAP50: {metrics.results_dict['metrics/mAP50(B)']:.4f}")
print("="*30)

# Bloco de Teste Prático (IMPORTANTE: Baixamos a confiança para 0.15)
print("Selecione a foto para testar (Arroz, Carne e Milho):")
uploaded = files.upload()
for fn in uploaded.keys():
    # conf=0.15 força o modelo a mostrar o que ele tem "dúvida", revelando o arroz e o milho
    results = model.predict(source=fn, conf=0.3)
    res_plotted = results[0].plot()
    cv2_imshow(res_plotted)
```